

Regression [MCQ] (Version : 0)

TEST

● **Correct Answer**

🕒 Answered in 6.32 Minutes

Question 1/30

How many predictors do we originally have in our dataset, and which of these are categorical in nature?

Hint: The `Field_ID` serves as a unique identifier for each field and does not provide any predictive value for modeling.

☐ 17 predictors; Location, Soil_type, and Crop_type are categorical.

☒ 16 predictors; Location, Soil_type, and Crop_type are categorical.

☐ 16 predictors; Location, Soil_type, and Plot_size are categorical.

☐ 18 predictors; Location, Soil_type, and Crop_type are categorical.

Explanation:

- **18 predictors; Location, Soil_type, and Crop_type are categorical:** This option correctly identifies the categorical variables but it counts all columns, including `Field_ID` and `Standard_yield`, as predictors.
- **17 predictors; Location, Soil_type, and Crop_type are categorical:** This option correctly identifies the categorical variables but includes an additional column (the ID field or target) in the predictor count.
- **16 predictors; Location, Soil_type, and Plot_size are categorical:** While this option gets the number of predictors correct, it incorrectly identifies `Plot_size` as categorical

when it is actually a numerical variable `float64`.

Question 2/30

The categorical features in our dataset need to be converted into a format suitable for modeling. After applying dummy variable encoding to these categorical features, how many independent variables do we now have?

Note: Set the `drop_first` parameter to True.

Hint: Remember to exclude `Field_ID`



29



17



16



31

Explanation:

31: This option did not consider subtracting `Field_ID` and the target variable after encoding the categorical variables.

18: This refers to the total number of columns in the dataset before dummy variable encoding is performed.

16: This is the number of predictor variables present in the dataset before applying dummy variable encoding.

Question 3/30

From our encoded dataset, which variable has the third highest absolute correlation with the `Standard_yield`, excluding the variable itself?



Annual_yield



Pollution_level



Crop_type_tea



Longitude

Explanation:

Crop_type_tea: This is the variable with the highest absolute correlation with Standard_yield after excluding the variable itself.

Annual_yield: this is the variable with the third highest positive correlation instead of absolute correlation.

Longitude: This is variable is further down the list with a correlation of 0.085343, placing it well below the third highest correlation.

Question 4/30

In order to fit an ordinary least squares regression model to our encoded data we would need to make sure that all our variables are numeric. Through dummy variable encoding, we converted our categorical variables to multiple int/boolean variables (depending on the environment you are working on). The sm.OLS() method itself does not inherently handle boolean data types implicitly.

In most cases, when fitting a model with sm.OLS(), it's essential to ensure that all features are numeric. If boolean columns are present, they should be explicitly converted to numeric types before fitting the model.

Suppose our columns converted to boolean datatypes, which of the following statements is true after converting the boolean columns in the dataset to integer datatype?



The boolean columns have been replaced with binary integer representations, with True converted to 1 and False converted to 0.



The boolean columns have been replaced with binary integer representations, with True converted to 0 and False converted to 1.



The boolean columns have been converted to a unique string datatype that is also an integer.



The boolean columns have been removed from the dataset.

Explanation:

The boolean columns have been removed from the dataset: There's no indication or instruction in the code that leads to the removal of columns.

The boolean columns have been replaced with binary integer representations, with True converted to 0 and False converted to 1: This is incorrect as it contradicts conversion logic between boolean and integer types which is that True is converted to 1, and False is converted to 0.

The boolean columns have been converted to a unique string datatype that is also an integer: The code clearly specifies the conversion of boolean columns to integer datatype, without any intermediate conversion to strings.

Question 5/30

Suppose we wish to determine which of the predictors in our dataset are statistically significant. We will follow the following steps:

- Fit a statsmodels regression model to predict `Standard_yield` using our encoded dataset.
- Extract p-values for each predictor from the fitted model to find the ones that are significant based on a given threshold.

Which of the following variables do we find to be statistically significant for predicting `Standard_yield`, based on a p-value threshold of 0.05 ($p\text{-value} < 0.05$)?



Elevation



Slope



pH



Soil_fertility

Explanation:

Slope, Elevation, and Soil_fertility are not listed among the significant variables in the output. This means that their p-values are above the 0.05 threshold.

Question 6/30

Create a correlation matrix using all the columns in our encoded dataset and display it as a heatmap.

Which of the following independent variables have a strong correlation with one another?

☐ Plot_size and Crop_type_tea

☐ Crop_type_cassava and Soil_type_Sandy

☒ Min_temperature_C and Elevation

☐ Pollution_level and Soil_type_Rocky

Explanation:

Plot_size and Crop_type_tea is incorrect because their correlation is 0.00

Pollution_level and Soil_type_Rocky is incorrect because their correlation is 0.01

Crop_type_cassava and Soil_type_Sandy is incorrect because their correlation is -0.01

Question 7/30

Now that we have analysed our variables, let's fit an ordinary least squares regression model using `statsmodels.formula.api` and then print the model summary. Construct the model using all the independent variables in our encoded dataset (excluding `Field_ID`).

Which of the following statements accurately describes the interpretation of the F-statistic in the context of our regression model?

☒ The F-statistic tests the overall significance of the regression model. A high F-statistic value with a corresponding low p-value indicates that the regression model is statistically significant, meaning that at least one of the independent

variables has a significant effect on the dependent variable.

☐ The F-statistic tests the overall significance of the regression model. A high F-statistic value with a corresponding high p-value indicates that the regression model is statistically significant, meaning that at least one of the independent variables has a significant effect on the dependent variable.

☐ The F-statistic tests the overall significance of the regression model. A low F-statistic value with a corresponding high p-value indicates that the regression model is statistically significant, meaning that at least one of the independent variables has a significant effect on the dependent variable.

☐

- The F-statistic tests the overall significance of the regression model. A low F-statistic value with a corresponding low p-value indicates that the regression model is not statistically significant, meaning that none of the independent variables have a significant effect on the dependent variable.

Explanation:

A low F-statistic value with a corresponding high p-value indicates that the regression model is statistically significant is incorrect because a low F-statistic value and a high p-value suggest that the regression model is not statistically significant, meaning that none of the independent variables have a significant effect on the dependent variable.

A high F-statistic value with a corresponding high p-value indicates that the regression model is statistically significant is incorrect because a high p-value typically indicates that the regression model is not statistically significant.

A low F-statistic value with a corresponding low p-value indicates that the regression model is not statistically significant is incorrect because a low p-

value typically indicates that the regression model is statistically significant, meaning that at least one of the independent variables has a significant effect on the dependent variable.

Question 8/30

This summary gives us an indication of possible multicollinearity present within our predictor variables. The presence of any correlation among predictors is detrimental to model quality because it tends to increase the standard error of the coefficients and it becomes difficult to estimate the effect of any one predictor variable on the response variable.

To avoid this, let's reduce the number of independent variables included in our model. Fit the model using the top seven variables that have a strong correlation to our dependent variable, namely, `Crop_type_tea`, `Soil_type_Volcanic`, `Pollution_level`, `Crop_type_coffee`, `Annual_yield`, `Location_Rural_Sokoto` and `Soil_type_Silt`.

After reducing the number of dependent variables accordingly, how did the model change?

☐ The model remained unchanged.

☒ The model worsened.

☐ It cannot be determined from the given information.

☐ The model improved.

Explanation:

"The model worsened" suggests that the model worsened after reducing the number of dependent variables. In the second regression model, the R-squared value increased from 0.406 to 0.756, indicating an improvement in the model's explanatory power. Therefore, this option is incorrect.

"The model remained unchanged." suggests that the model remained unchanged after reducing the number of dependent variables. This is also incorrect because there was a noticeable change in the R-squared value, which increased from 0.406 to 0.756. A higher R-squared value indicates a better fit of the model to the data, demonstrating an improvement. Therefore, this option is incorrect.

"It cannot be determined from the given

information." suggests that it cannot be determined from the given information whether the model improved or not. However, based on the provided regression results, it is evident that the model did improve. The increase in the R-squared value from 0.406 to 0.756 indicates a better fit of the model to the data, resulting in an improvement. Therefore, this option is incorrect.

Question 9/30

Let's evaluate our model's results. Generate a scatter plot of the residuals against the fitted values allowing us to visually inspect whether the residuals have constant variance and are distributed randomly around the zero residual line.

What does the scatter plot tell us?

☐ The model suffers from multicollinearity.

☐ There is no linear relationship between the dependent and independent variables.

☒ The plot indicates homoscedasticity as residuals have a constant variance and are distributed randomly around the zero residual line

☐

- The model has perfect predictive accuracy.

Explanation:

"The model has perfect predictive accuracy." is incorrect because while the plot indicates that the model's errors have a constant variance and are distributed randomly around the zero residual line, it does not necessarily imply perfect predictive accuracy. The scatter plot's indication of homoscedasticity is only one aspect of evaluating a model's performance, and other factors such as the R-squared value and significance of coefficients should also be considered.

"There is no linear relationship between the dependent and independent variables." is incorrect because the model's errors having a constant

variance and being distributed randomly around the zero residual line does not imply the absence of a linear relationship between the variables.

Homoscedasticity refers to the constant variance of residuals, while a linear relationship pertains to the association between the dependent and independent variables.

"The model suffers from multicollinearity." is incorrect because this conclusion cannot be drawn solely from the information provided about the scatter plot. Multicollinearity refers to the presence of high correlation among independent variables, which can affect the model's coefficients and interpretations. The scatter plot's indication of homoscedasticity does not directly relate to multicollinearity.

Question 10/30

If a linear regression model indicated heteroscedasticity, which of the following actions could be considered to address this issue?

☐ Applying transformations to the independent variables to better fit the linear relationship.

☐ Implementing weighted least squares regression to give less emphasis to observations with higher variance in residuals.

☒ All of the above.

☐ Removing outliers from the dataset to reduce the impact of extreme values on the variance of residuals.

Explanation:

"Applying transformations to the independent variables to better fit the linear relationship." suggests applying transformations to the independent variables to better fit the linear relationship. While transforming variables can sometimes help address heteroscedasticity, it is not the only solution and may not always be effective.

Additionally, transforming variables can make interpretation more complex and may not fully resolve the issue of heteroscedasticity. Therefore, this option is not necessarily the only action that could be considered to address heteroscedasticity, making this option alone incorrect.

"Implementing weighted least squares regression to give less emphasis to observations with higher variance in residuals" proposes removing outliers from the dataset to reduce the impact of extreme values on the variance of residuals. While removing outliers can help mitigate heteroscedasticity to some extent, it may not completely resolve the issue, especially if other underlying factors contribute to the heteroscedasticity. Additionally, removing outliers without careful consideration can lead to loss of valuable information and potential bias in the analysis. Therefore, while removing outliers can be one strategy, it is not the only action that could be considered to address heteroscedasticity, making this option alone incorrect.

Therefore, the correct option is "All of the above," as it states that a combination of actions, including applying transformations to the independent variables and implementing weighted least squares regression, along with removing outliers, could be considered to address the issue of heteroscedasticity.

Question 11/30

Our first step in the feature engineering involves creating a new feature, `Temperature_Range`, and scaling features using `StandardScaler`. The difference between `Min_temperature_C` and `Max_temperature_C` is the temperature range.

Given the code block in the notebook, which option correctly completes the feature engineering step of creating the `Temperature_Range` column?

Hint: Insert the correct code snippet into the appropriate location within the code block to run it.

☐ `dataset[["Temperature_Range"]] =
dataset[["Max_temperature_C"]] -
dataset[["Min_temperature_C"]]`

☐ `dataset["Temperature_Range"] =
dataset[["Min_temperature_C", "Max_temperature_C"]].min(axis=1)`



```
dataset =  
dataset.assign(Temperature_Range =  
dataset['Max_temperature'] -  
dataset['Min_temperature'])
```



```
dataset['Temperature_Range'] =  
dataset['Max_temperature_C'] -  
dataset['Min_temperature_C']
```

Explanation:

- `dataset['Temperature_Range'] = dataset[['Min_temperature_C', 'Max_temperature_C']].min(axis=1)`

This option calculates the minimum value between `Min_temperature_C` and `Max_temperature_C` for each row. The operation finds the lowest temperature out of the minimum and maximum temperatures, which is not the range. The temperature range should be the difference (i.e., subtraction) between the maximum and minimum temperatures, representing how much temperature varies.

- `dataset = dataset.assign(Temperature_Range = dataset['Max_temperature'] - dataset['Min_temperature'])`

This code snippet contains a subtle mistake in the column names; it refers to `Max_temperature` and `Min_temperature` instead of `Max_temperature_C` and `Min_temperature_C`. This mistake will lead to an error if the specified columns do not exist, or it will calculate the range using the wrong columns if they do exist by chance. Correct feature engineering requires accurate column references.

- `dataset[['Temperature_Range']] = dataset[['Max_temperature_C']] - dataset[['Min_temperature_C']]`

Although this approach technically calculates the difference between the maximum and minimum temperatures, using double brackets (`[[ ]]`) on the left-hand side of the assignment is unnecessary and can be misleading. This syntax suggests the assignment to a DataFrame instead of creating a new column. While Python and pandas are tolerant of this syntax and it will

work, it is not the conventional way to add a new column to a DataFrame. The correct method is to use single brackets (`[ 'Temperature_Range' ]`) on the left side to assign a new column.

Question 12/30

Consider a scenario where we decide to employ LASSO regression to identify predictive features associated with our dependent variable.

We implement the Python code block provided in the notebook.

What is the purpose of the `LassoCV(cv=5)` constructor parameter `cv=5`?

☒ It indicates that 5-fold cross-validation should be used to select the best regularisation parameter.

☐ It specifies that the LASSO model should use a 5-degree polynomial feature expansion.

☐ It defines that the model should only select 5 features at most.

☐ It sets the penalty coefficient to 5, increasing the regularisation strength.

Explanation:

- **It sets the penalty coefficient to 5, increasing the regularisation strength.** Incorrect because the `cv` parameter in `LassoCV(cv=5)` does not set the penalty coefficient. Instead, `cv` stands for cross-validation and specifies the number of folds used in cross-validation. The penalty coefficient, which determines the strength of the regularisation, is automatically chosen through cross-validation in `LassoCV`, not manually set to a specific value like 5.
- **It specifies that the LASSO model should use a 5-degree polynomial feature expansion.** This option misunderstands the role of the `cv` parameter in `LassoCV`.

The `cv` parameter has nothing to do with feature expansion or the degree of polynomial features. Polynomial feature expansion is a separate preprocessing step where original features are raised to powers up to a specified degree to capture non-linear relationships. The `cv=5` parameter only informs the cross-validation process for selecting the best regularisation parameter, not feature manipulation.

- **It defines that the model should only select 5 features at most.** Again, this interpretation is incorrect because the `cv` parameter does not dictate the number of features to select. LASSO regression inherently performs feature selection by shrinking some coefficients to zero, depending on the regularisation strength. The number of features selected depends on the outcome of the regularisation process, which is influenced by the regularisation strength (chosen via cross-validation in `LassoCV`), not a predefined limit on the number of features.

Question 13/30

In trying to address multicollinearity in our dataset, we also decide to implement Ridge regression. After understanding that Ridge regression applies an L2 penalty to the coefficients to reduce their magnitude without setting them to zero, we decide to use `RidgeCV` for applying Ridge regression with cross-validation to select the optimal penalty strength. Given the snippet of code in the notebook, which parameter correctly adjusts the strength of the regularisation applied to the model?

Hint: Insert the correct code snippet into the appropriate location within the code block to run it.

☒ `alphas=[0.001, 0.01, 0.1, 1, 10, 100]`

☐ `fit_intercept=True`

☐ `cv=[0.001, 0.01, 0.1, 1, 10, 100]`

☐ `scoring='neg_mean_squared_error'`

Explanation:

- `cv=[0.001, 0.01, 0.1, 1, 10, 100]`

The `cv` parameter in `RidgeCV` does not adjust the strength of regularisation. Instead, it specifies the cross-validation splitting strategy, which determines how the data is split during the cross-validation process. It should be an integer specifying the number of folds in a `(Stratified)KFold`, a cross-validation splitter instance, or an iterable yielding train/test splits. Providing alpha values (used for regularisation strength) here is a misuse of the parameter.

- `fit_intercept=True`

The `fit_intercept` parameter specifies whether to calculate the intercept for this model. If set to false, no intercept will be used in calculations (i.e., data is expected to be centered). This parameter does not influence the strength of regularisation directly but rather adjusts how the model accounts for the intercept. It's a structural model choice, not a regularisation strength adjustment.

- `scoring='neg_mean_squared_error'`

The `scoring` parameter determines the metric to use for evaluating the performance of cross-validation models. In the context of `RidgeCV`, it might influence the selection of the best model under cross-validation but does not adjust the regularisation strength itself. This parameter is about model evaluation, not the configuration of the regularisation process.

Question 14/30

Given our dataset includes variables such as `Elevation` and `Slope`, and considering the potential interactions between these variables might impact crop yield, we aim to capture both these interactions and possible non-linear relationships. Which of the following methods is specifically designed to create a quadratic interaction term without including an intercept in the feature set?

HINT: Consider utilising `sklearn`'s feature transformation tools for this purpose.



`poly = PolynomialFeatures(degree=2,
include_bias=False)`



`poly = PolynomialFeatures(degree=3,
include_bias=True)`



```
poly = PolynomialFeatures(degree=1,  
include_bias=False)
```



```
poly = PolynomialFeatures(degree=2,  
include_bias=True)
```

Explanation:

- **poly = PolynomialFeatures(degree=1, include_bias=False)**: This option is incorrect because it specifies a polynomial degree of 1, which means it does not create interaction terms between features or any polynomial terms beyond the first degree (linear scaling). The essence of the question was to capture both quadratic (second degree) interactions and possible non-linear relationships between the variables, which a degree of 1 cannot achieve.
- **poly = PolynomialFeatures(degree=3, include_bias=True)**: This choice is incorrect for two reasons. Firstly, it specifies a polynomial degree of 3, which means it creates cubic terms. The question, however, asks for quadratic interaction terms, implying a degree of 2. Secondly, setting **include_bias=True** includes an intercept term in the output feature set, which the question explicitly wants to avoid. The focus is on quadratic interactions without an added intercept term.
- **poly = PolynomialFeatures(degree=2, include_bias=True)**: While this option does specify the correct polynomial degree (2), making it capable of creating the desired quadratic interaction terms, it also sets **include_bias=True**, which leads to the inclusion of an intercept term in the feature set. The intercept term is not desired in this specific scenario as per the question's requirement, making this option incorrect.

Question 15/30

After implementing Ridge regression to address multicollinearity and prevent overfitting in our model we need to interpret the coefficients to understand the impact of each feature on the standard yield.

Based on the output of the Ridge regression coefficients in the code block given in the notebook, which

statement is true regarding the impact of each feature on the standard yield?

☐

Feature 1 has the highest positive impact, followed by Feature 2 and Feature 3.

☐

The coefficients are inconclusive; further analysis is needed to determine the impact of each feature.

☐

Feature 3 has the highest positive impact, followed by Feature 1 and Feature 2.

☒

Feature 2 has the highest positive impact, while Feature 3 and Feature 1 have negative impacts.

Explanation:

- **Feature 1 has the highest positive impact, followed by Feature 2 and Feature 3:** Ridge regression outputs coefficients that directly measure the impact of each feature on the target variable, after adjusting for multicollinearity and penalizing large coefficients to prevent overfitting. If this option is incorrect, it implies that the interpretation of the Ridge regression coefficients does not support the conclusion that Feature 1 has the highest positive impact. In Ridge regression, the magnitude and sign of each coefficient indicate the direction and strength of a feature's impact. A correct interpretation of these coefficients should directly reflect the mathematical relationship outlined by the model, which, in this case, does not rank Feature 1 as having the highest positive impact based on the question's context.

- **Feature 3 has the highest positive impact, followed by Feature 1 and Feature 2:**

This option is incorrect because it misinterprets the coefficients' values from the Ridge regression model. The essence of Ridge regression is to provide clear insights into how each independent variable (feature) influences the dependent variable (target), considering the penalty on the size of coefficients to manage multicollinearity and overfitting. If Feature 3 is stated to have the highest positive impact contrary to the

model's output, it signifies a misunderstanding of how to read and interpret these coefficients. Each coefficient value reflects the change in the target variable for a one-unit change in the corresponding feature, while holding other features constant. The mathematical output from Ridge regression contradicts the assertion made in this option.

- **The coefficients are inconclusive; further analysis is needed to determine the impact of each feature:**

This statement would be incorrect because Ridge regression coefficients are specifically designed to be interpretable in terms of the impact of each feature on the outcome, even when dealing with multicollinearity. Saying that the coefficients are inconclusive overlooks the fundamental purpose of Ridge regression, which includes regularisation to ensure that the coefficients remain interpretable despite high correlations among features. While it's true that further analysis is always beneficial for a comprehensive understanding, the Ridge regression model provides conclusive insights into the direction and magnitude of each feature's impact based on its coefficients. Therefore, dismissing the coefficients as inconclusive misrepresents the utility and application of Ridge regression in statistical modeling and analysis.

Question 16/30

Train a decision tree with the following specifications:

- Using our **previously encoded dataset**, split the data into dependent and independent variables using all the features except for **Standard_yield** and **Field_ID** as independent variables.
- Split the data into training and testing data.
- Use the **DecisionTreeRegressor** to fit a model using a **max_depth' of 2 and a random_state` of 42**.

Using the trained Decision Tree Regressor model, make a prediction for **y** given the following x-values:

[864.66138, -8.12890218821531, -8.311822719284072, 16.274624300000003, 1237.7200000000003, -3.4100000000000006, 36.410000000000004, 16.5, 0.682, 6.7863323423108195, 0.09379352739936421, 1.4300000000000002, 0.8264890400277934, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.1, 0.0, 0.0, 1.1, 0.0, 0.0, 0.0, 0.0, 0.0]

What is the value of the predicted y?



0.48494414

☐ 0.8050340

☐ 0.6654377

☐ 0.3250077

Explanation:

After running the correct code to make a prediction for y given the provided x -values, the result we get is 0.48494414 therefore the other options are incorrect.

Question 17/30

Based on the model in the previous question, what is the value of our RMSE?

☐ 0.0658

☐ 0.8810

☒ 0.0881

☐ 0.5656

Explanation:

After running the correct code to calculate RMSE using `mean_squared_error(y_pred, y_test)`, the result we get is 0.0881 therefore the other options are incorrect.

Question 18/30

Which of the following statements is correct about our RMSE?

☐ An RMSE of 0.0881 suggests that the predicted values are correct 8.81% of the time.

☐ An RMSE value of 0.0881 suggests that, at most, the predicted values deviate by approximately 0.08 units from the actual values.

☒ An RMSE of 0.0881 suggests that, on average, the predicted values are off by approximately 0.0881 units

☐ An RMSE of 0.0881 indicates a perfect fit of the model to the training data.

Explanation:

"An RMSE value of 0.0881 indicates that, at most, the predicted values deviate by approximately 0.08 units from the actual values" is incorrect because RMSE measures the average magnitude of the errors between predicted and actual values, not the maximum deviation.

"An RMSE of 0.0881 indicates a perfect fit of the model to the training data" is incorrect because it claims that an RMSE of 0.0881 indicates a perfect fit of the model to the training data. However, this statement is incorrect because an RMSE of 0.0881 does not imply a perfect fit. RMSE represents the standard deviation of the residuals, so a value of 0.0881 would mean that there is some variability in the errors, therefore not a perfect fit.

"An RMSE of 0.0881 indicates that the predicted values are correct 8.81% of the time" is incorrect because RMSE does not directly represent the percentage of correct predictions. Instead, it quantifies the average magnitude of the errors between predicted and actual values.

Question 19/30

What is the likely effect of increasing the `max_depth` parameter in a Decision Tree model?

☐ Higher `max_depth` values may lead to decreased model complexity and a lower risk of overfitting.



Higher max_depth values may lead to increased model complexity and a higher risk of overfitting.



Higher max_depth values reduce the likelihood of capturing complex relationships in the data.



Lower max_depth values may lead to increased model complexity and a higher risk of underfitting.

Explanation:

"Lower max_depth values may lead to increased model complexity and a higher risk of underfitting." is incorrect because lower max_depth values typically result in simpler models with less depth and fewer splits, which may lead to a higher risk of underfitting due to the model's inability to capture complex patterns in the data.

"Higher max_depth values may lead to decreased model complexity and a lower risk of overfitting." is incorrect because higher max_depth values often result in more complex models with deeper trees and more splits, which may lead to a higher risk of overfitting as the model captures noise and outliers in the training data.

"Higher max_depth values reduce the likelihood of capturing complex relationships in the data." is incorrect because higher max_depth values actually increase the likelihood of capturing complex relationships by allowing the decision tree to create more intricate splits and patterns.

Question 20/30

Let's attempt to enhance our model's performance by setting the max_depth hyperparameter to 5.

True or false? The decision tree model was improved by fitting it with a max_depth parameter of 5.



False



True

Explanation:

"False" suggests that the decision tree model was not improved by fitting it with a `max_depth` parameter of 5. However, since the RMSE was reduced from 0.0881 to 0.0633, it indicates an improvement in model performance. A lower RMSE value signifies better predictive accuracy, implying that the model's ability to predict outcomes has improved.

Therefore, the correct option is "True" because the reduction in RMSE from 0.0881 to 0.0633 indicates an improvement in the decision tree model's performance, suggesting that fitting it with a `max_depth` parameter of 5 led to an enhancement in predictive accuracy.

Question 21/30

Consider the following approach to implement a bootstrap aggregation (bagging) for predicting the `Standard_yield` based on features `Elevation`, `Slope`, `Soil_fertility`, and `Pollution_level`. Given are the steps and part of the Python code implementing this method. Our task is to identify the correct piece of code that completes the implementation.

The steps for the implementation are as follows:

1. Create an empty list named `predictions` to store predictions from each bootstrap sample.
2. Generate `n_bootstrap_samples` bootstrap samples from the original dataset.
3. For each bootstrap sample, fit a linear regression model and predict the `Standard_yield` on the entire dataset.
4. Store each set of predictions in the predictions list.
5. Average the predictions across all bootstrap samples to obtain the final bagged prediction.
6. Compute and print the mean squared error (MSE) to evaluate the performance of the bagged model.

Given, in the notebook, is the code block with a missing part that needs to be completed

Which code should replace the `# MISSING PART HERE` section to correctly implement the averaging of predictions and computation of the mean squared error?

☐

```
bagged_prediction =  
np.median(predictions, axis=0)  
  
mse_bagged = mean_squared_error(y,  
bagged_prediction)
```

☐

```
bagged_prediction = np.mean(predictions,  
axis=1)  
  
mse_bagged = mean_squared_error(y,  
bagged_prediction)
```



```
bagged_prediction =  
np.average(predictions, axis=1)  
  
mse_bagged = mean_squared_error(y,  
bagged_prediction)
```



```
bagged_prediction = np.mean(predictions,  
axis=0)  
  
mse_bagged = mean_squared_error(y,  
bagged_prediction)
```

Explanation:

- **bagged_prediction = np.average(predictions, axis=1); mse_bagged = mean_squared_error(y, bagged_prediction)**

The `np.average(predictions, axis=1)` computes the average across the wrong axis. Given that `predictions` is a list where each element is an array of predictions for the entire dataset, averaging across `axis=1` would imply trying to average across the individual predictions within each bootstrap sample, which does not align with the dimensions of `predictions`. We want to average the predictions of the same data point across all bootstrap samples, which corresponds to averaging across `axis=0`.

- **bagged_prediction = np.median(predictions, axis=0); mse_bagged = mean_squared_error(y, bagged_prediction)**

This option uses the median instead of the mean to aggregate predictions across bootstrap samples. While taking the median can be a robust aggregation method, the instruction specifies averaging the predictions to obtain the final bagged prediction. Averaging is done through computing the mean, not the median. Thus, using the median does not follow the specified steps for bagging in this context.

- **bagged_prediction = np.mean(predictions, axis=1); mse_bagged = mean_squared_error(y, bagged_prediction)**

Similar to the first incorrect option, this command attempts to compute the mean across `axis=1`. This is incorrect for the same reason: it misinterprets the structure of `predictions`. The `predictions` list needs to be averaged across `axis=0` to average each data point's predictions across all bootstrap samples. Averaging across `axis=1` does not make sense in this context and will likely result in an error or incorrect computation, as the shape of `predictions` does not support this operation in the intended manner.

Question 22/30

Given the provided code snippet that applies a `RandomForestRegressor` to a dataset, which parameter in the `RandomForestRegressor` constructor is crucial for implementing the random subspace method by allowing the algorithm to select a random subset of features for each split?



`max_features`



`random_state`



`max_depth`



`n_estimators`

Explanation:

`max_depth`: This parameter limits the maximum depth of the tree. A tree's depth is the longest path from the root node to a leaf. Limiting the depth of a tree can help prevent overfitting by reducing the model's complexity. However, it does not influence which or how many features are considered for each split, which is what the random subspace method is about.

`random_state`: The `random_state` parameter is used to seed the random number generator. This is useful for creating reproducible results, as it ensures that the random elements of the algorithm (like choosing features if `max_features` is less than the number of features) produce the same result every time the algorithm is run with the same data and parameter settings. While it influences

randomness, it doesn't control the number of features to consider at each split.

n_estimators: This parameter specifies the number of trees in the forest. Essentially, it controls the ensemble's size but has no direct influence on the feature selection process for individual trees. A larger number of trees can increase the model's performance up to a certain point by reducing overfitting through averaging more trees' predictions. However, it does not affect the implementation of the random subspace method since it doesn't determine how features are selected for splits within each tree.

Question 23/30

Consider the theoretical setup for a stacking ensemble model designed for a regression task. The first layer of this model includes three different types of regression models: linear regression, ridge regression, and a support vector machine (SVM) with a linear kernel. The second layer, or the final estimator, uses a linear regression model to combine the predictions from the first layer. The goal is to theoretically predict **Standard_yield** based on features such as **Elevation**, **Slope**, **Soil_fertility**, and **Pollution_level**, with the intention to evaluate the model's hypothetical performance using the Mean Squared Error (MSE).

Given the theoretical code snippet in the notebook that outlines this stacking ensemble model's setup, what should replace the _____ in the code to correctly configure the SVM with a linear kernel as part of the base learners in the stacking model?

☐ lin

☐ degree=1

☒ linear

☐ sigmoid

Explanation:

- **lin:**

This option is incorrect because **lin** is not a recognised kernel option within the SVM algorithm in scikit-learn. The SVM algorithm requires specific strings to identify the kernel type, such as **linear**, **poly**, **rbf**, and **sigmoid**. Since **lin** does not match any of these recognized kernel types, it will result

in an error.

- **sigmoid:**

Choosing a **sigmoid** kernel for the SVM means the decision boundary is defined using a sigmoid function. While the sigmoid kernel can model nonlinear relationships, it is not equivalent to a linear kernel and typically performs differently. The **sigmoid** kernel transforms the input space in a way that is not linear, which does not align with the requirement for a linear kernel in the question.

- **degree=1:**

This option suggests setting the polynomial degree of the kernel to 1, which might seem like it would create a linear relationship. However, this configuration is applicable only when the kernel is explicitly set to **poly** for polynomial. Without specifying **kernel='poly'**, setting **degree=1** alone does not make sense and would not correctly configure the SVM to use a linear kernel. Furthermore, the degree parameter does not apply to the linear kernel.

Question 24/30

Consider the Python code snippet given in the notebook, that aims to implement a 5-fold cross-validation scheme to estimate the accuracy of a ridge regression model. This model uses bootstrapped samples within each fold to predict **Standard_yield** and calculates the average Mean Squared Error (MSE) across all folds.

Which of the following options correctly fill in the blanks to ensure the code correctly implements the described functionality?

☐ `scoring='mean_absolute_error', cv=5`

☒ `scoring='neg_mean_squared_error', cv=5`

☐ `scoring='r2', cv=10`

☐ `scoring='neg_mean_squared_error',
cv='Bootstrap'`

Explanation:

- **scoring='r2', cv=10**: The **scoring** parameter is set to **'r2'**, which measures the coefficient of determination, not the Mean Squared Error (MSE). Since the question specifies that the aim is to calculate the average MSE, using **r2** as the scoring metric does not align with this objective. Additionally, **cv=10** specifies a 10-fold cross-validation, whereas the question clearly states that a 5-fold cross-validation scheme is to be implemented.
- **scoring='mean_absolute_error', cv=5**: The **scoring** parameter here is **'mean_absolute_error'**. This specifies the Mean Absolute Error (MAE) as the metric for evaluating the model's performance. However, the question asks for the Mean Squared Error (MSE), not the MAE. Therefore, this option does not correctly follow the instructions for calculating the average MSE. The **cv=5** part is correct as per the question's requirement for a 5-fold cross-validation.
- **scoring='neg_mean_squared_error', cv='Bootstrap'**: While the scoring parameter is correctly set to **'neg_mean_squared_error'**, specifying **cv='Bootstrap'** is incorrect for multiple reasons. First, the **cross_val_score** function in scikit-learn does not accept a string value like **'Bootstrap'** for the **cv** argument. The **cv** parameter expects either an integer, specifying the number of folds in a (Stratified)KFold, a CV splitter, or an iterable yielding (train, test) splits as arrays of indices. Second, the question explicitly mentions a 5-fold cross-validation scheme, and using bootstrapping would require a different approach not covered by simply setting **cv='Bootstrap'**. Bootstrapping involves resampling with replacement, which is different from the k-fold cross-validation method and is not directly supported by the **cross_val_score** function's **cv** argument in this manner.

Question 25/30

Consider the code snippet that extracts and prints the feature importances from a trained random forest regressor model. The model is used to predict **Standard_yield** based on various features. The code utilises

the `feature_importances_` attribute of the random forest model to obtain importance scores for each feature.

Which of the following statements best describes the purpose and outcome of the provided code snippet?



The code identifies and prints the importance scores for each feature in the random forest model, indicating how much each feature contributes to the model's ability to predict `Standard_yield`. Higher scores suggest a greater contribution to the prediction.



The code computes the correlation between each feature and the target variable `Standard_yield`, printing a list of features sorted by their correlation coefficients to identify the most relevant predictors.



The code counts the number of times each feature is used to split the data across all trees in the random forest, thereby determining each feature's importance in predicting `Standard_yield`.



The code calculates and prints the coefficient values for each feature used in the random forest, model to predict `Standard_yield`, indicating the strength and direction of the relationship between each feature and the target variable.

Explanation:

- **The code counts the number of times each feature is used to split the data across all trees in the random forest, thereby determining each feature's importance in predicting `Standard_yield`.**

This statement misunderstands the nature of `feature_importances_`. While the underlying principle of a feature's importance can relate to how often it's used in splits across the forest's trees, `feature_importances_` doesn't directly count split occurrences. Instead, it quantifies the *impact* of each feature on the model's output, specifically how much each feature decreases the weighted impurity across all trees, which is a more nuanced measure than

mere count of splits.

- **The code calculates and prints the coefficient values for each feature used in the random forest model to predict `Standard_yield`, indicating the strength and direction of the relationship between each feature and the target variable.**

This option confuses the concept of feature importance with regression coefficients, which are not the same. Regression coefficients, indicating the strength and direction of the relationship between each feature and the target variable, are a characteristic of linear models (like linear regression), not random forest regressors. Random forest models do not produce coefficients that tell us the direction (positive or negative) of the relationship between a feature and the target variable; instead, they provide importance scores that reflect how crucial a feature is for making accurate predictions, without indicating directionality.

- **The code computes the correlation between each feature and the target variable `Standard_yield`, printing a list of features sorted by their correlation coefficients to identify the most relevant predictors.**

This option incorrectly assumes that `feature_importances_` calculates correlation coefficients between features and the target variable. In reality, correlation coefficients measure the linear relationship between variables and are completely separate from the concept of feature importance in machine learning models like random forests. The feature importance scores derived from a random forest model are based on how changes in a feature value lead to changes in the model's accuracy or impurity reduction, rather than on calculating linear correlation coefficients.

Question 26/30

We are required to write a function named `train_rf_model` that trains and tests a random forest model on a given dataset. Our function should do the following:

- Take a `RandomForestRegressor` object (with any desired hyperparameters set) as input,
- Separate the features `x` and target `y` dataframes
- Split the data into training and testing sets - use a test size of `20%` and a random state of `42` for reproducibility
- Fit the model to the training data
- Make predictions on the testing set
- Return the trained model, the R-squared score, and the Mean Squared Error (MSE) of the test set predictions.

1. Implement the function outlined above.
2. Using the function, train a random forest model on our dataset with `random_state` set to `42`, and `max_depth=15`, while leaving all other hyperparameters at their defaults. Use all the features available in the encoded dataset for this task. What are the R-squared and MSE scores for the model on the test data?

☐ R2: 0.8196, MSE: 0.0500

☒ R2: 0.9586, MSE: 0.0006

☐ R2: 0.5555, MSE: 0.2345

☐ R2: 0.0059, MSE: 0.6198

Explanation:

R2: 0.9618, MSE: 0.0005 matches the output from the provided code, with the R2 score and MSE values rounded to four decimal places. We observe a high R2 score and a low MSE value which indicates a well-performing model on the test dataset.

The rest of the options do not represent the combination of the R-squared (R2) and Mean Squared Error (MSE) scores gotten after running the provided code.

Question 27/30

We want to examine how our data will fit to a random forest model when we tune the number of trees. We want to train and compare two random forest models with the same dataset as in the previous exercise. The first model should be trained with `150` trees, and the second model with `200` trees. Both models should use the default hyperparameters for all other settings, apart from a `random_state` of `42` to ensure reproducibility. After evaluating both models on the test set, how does the error differ between the two models?



The model with 200 trees showed a very slight decrease in error compared to the model with 150 trees.



There was no change in the error.



The model with 200 trees showed a significant decrease in error compared to the model with 150 trees.



The error increased when the number of trees was increased from 150 to 200.

Explanation:

The error increased when the number of trees was increased from 150 to 200: The output indicates a decrease, while slight, in the MSE when the number of trees is increased from 150 to 200, not an increase.

The model with 200 trees showed a significant decrease in error compared to the model with 150 trees.: While there is a decrease in error when the number of trees is increased from 150 to 200, the change in MSE is very slight and not significant.

There was no change in the error: Stating there was no change in error is inaccurate because the output demonstrates a change in the MSE, even though the difference is minor.

Question 28/30

Which of the following is a possible effect of increasing the number of trees in a random forest regression model?



Increasing the number of trees always improves the model's predictive ability



Increasing the number of trees increases the model's predictive ability up to a certain point, after which additional trees

do not significantly impact performance.

☐

Increasing the number of trees significantly decreases the model's predictive ability

☐

The number of trees in a random forest model has no impact on the model's predictive ability.

Explanation:

Increasing the number of trees always improves the model's predictive ability: Continuously adding more trees does not always translate to better predictive performance as the performance improvement plateaus after a certain number of trees.

Increasing the number of trees significantly decreases the model's predictive ability: Increasing the number of trees in a random forest improves its predictive ability (up to a point) or has a negligible effect beyond that point, but does not decrease it.

The number of trees in a random forest model has no impact on the model's predictive ability: This is incorrect because increasing the number of trees can have a positive impact on a random forest model's predictive ability.

Question 29/30

Following the training of our random forest models, we decide to analyse the feature importance scores provided by the model built using 200 trees. Our aim is to identify which features the model considers most significant in predicting the target variable.

Which of the following does the model consider to be the top 3 most significant features in predicting Standard_yield?

☐

Soil_fertility, Rainfall, Slope

☐

Elevation, Soil_fertility, pH

☐

pH, Rainfall, Location_Rural_Hawassa



Rainfall, Crop_type_tea, Latitude

Explanation:

Soil_fertility, Rainfall, Slope: This option incorrectly includes Soil_fertility and Slope, which are not among the top 3 features according to the model's feature importances.

pH, Rainfall, Location_Rural_Hawassa: This option incorrectly includes pH and Location_Rural_Hawassa, which are not among the top 3 features according to the model's feature importances.

Elevation, Soil_fertility, pH: All these three features are not listed among the top 3 features according to the model's feature importances.

Question 30/30

Which of the following initialised random forest models will allow access and calculation of the Out-of-Bag (OOB) score for performance evaluation without requiring a separate validation set?



RandomForestRegressor(n_estimators=100,
max_depth=10, min_samples_split=2,
random_state=42)



RandomForestRegressor(n_estimators=200,
max_depth=5, min_samples_leaf=4,
oob_score=False)



RandomForestRegressor(n_estimators=150,
max_depth=None, oob_score=True)



RandomForestRegressor(n_estimators=120,
max_features='sqrt', min_samples_leaf=2)

Explanation:

RandomForestRegressor(n_estimators=100,
max_depth=10, min_samples_split=2,

random_state=42): Although this model specifies various parameters, it does not explicitly enable the oob_score. By default, oob_score is set to False, meaning this model will not calculate the OOB score for performance evaluation.

RandomForestRegressor(n_estimators=200, max_depth=5, min_samples_leaf=4, oob_score=False): This option explicitly sets oob_score to False, indicating that the model is not configured to calculate or allow access to the OOB score.

RandomForestRegressor(n_estimators=120, max_features='sqrt', min_samples_leaf=2): Although this model specifies various parameters, it does not explicitly enable the oob_score. By default, oob_score is set to False, meaning this model will not calculate the OOB score for performance evaluation.